Team Number:	apmcm25306486
Problem Chosen:	A

2025 APMCM summary sheet

Unitree Robotics, a leading company in China, has developed the G1 robot, which has been widely applied worldwide. Achieving reliable task execution for humanoid robots requires addressing several key problems, including arm control, walking kinematics, and energy management. This paper investigates these aspects and provides corresponding modeling and analysis.

For Problem 1, we need to use the given joint angles to compute the end-effector position and verify motor safety. A **standard Denavit－Hartenberg model(D-H model)** is constructed by assigning coordinate frames to each link and determining the corresponding D－H parameters and inter-frame transformation matrices based on the robot's geometry. The transformation from the base frame to the end-effector frame is then obtained, yielding an end-effector position of (**126**.**75mm**, **73**.**18mm**, **84**.**50mm**). Finally, based on the torque-balance principle, we develop a **motor safety model**, confirming that the motion above is executed within safe limits.

For Problem 2, the time-varying joint angles of the robot during walking are needed to obtained. The global velocity profile of the robot's CoM is generated based on **sinusoidal speed planning**. Then, the walking motion is divided into single-support and double-support phases, for which models are established using the **inverted pendulum** and **quintic polynomial fitting** to obtain accurate velocity trajectories. Furthermore, inverse kinematics is applied to construct equations relating the leg joint parameters. Finally, ankle joint nodes are used to transform the problem into an optimization task. By combining **global sampling** with **local search** and solving via **the Sequential Quadratic Programming algorithm**, the joint angles at given times are determined, with the maximum knee angle occurring at **1**.**744s**.

For Problem 3, we aim to obtain the robot's joint angles as functions of time during combined arm－torso motions. Based on separate analyses of torso rotation and circular arm trajectories, we establish an **upper-limb kinematic model** and a **CoM-based trajectory planning model**, from which the corresponding joint angles are obtained through spatial geometry. Subsequently, a **dynamic compensation model** was constructed based on geometric and kinematic analysis to determine the leg joint angles required to maintain balance during this motion.

For Problem 4, we establish a **single-objective optimization model** that **minimizes energy usage** while satisfying constraints on joint angle limits, motor torque safety, voltage limits, motion stability, and performance quality. Using the **Lagrangian energy formulation**, we derive the relationship among joint angles, motor torque, and motor power, and compute the total energy consumption of the original motion to be approximately **77**.**46** J. The nonlinear constrained optimization problem is solved using the SQP algorithm, yielding an improved motion plan that reduces energy consumption about **15**.**68**% compared with the original plan.

Finally, the proposed models integrate various aspects, offering broad application potential. Future work may incorporate sensors for tasks like obstacle avoidance and cooperative control.

Keywords: Standard D−H Model　SQP algorithm　Inverse Kinematics

Contents

1. Introduction **1**
1.1 Problem Restatement 1
1.2 Literature Review. 1
1.3 Problem analysis 2
1.4 Our Work 3
2. Assumptions and Notations **4**
2.1 Assumptions 4
2.2 Notations 4
3. Problem 1: Model Construction and Solution **4**
3.1 Position Determination Model 5
3.2 Motion Safety Verify Model 7
3.3 Model Results 8
4. Problem2: Model Construcion and Solution **9**
4.1 Global Assumptions 9
4.2 Sinusoidal-Profile-Based Global CoM Motion Planning Model 10
4.3 Trajectory Replanning Model Based on Hybrid Dynamics. 11
4.3.1 *Single-Support Phase (SSP) Mechanism Model* 11
4.3.2 *Double-Support Phase Mechanism Model* 13
4.4 6-DoF Serial Manipulator Inverse Kinematics Model 14
4.4.1 *D-H Method Construction* 14
4.4.2 *Inverse Kinematics* 14
4.5 Model Solution 15
5. Problem3: Model Construcion and Solution **17**
5.1 Model Assumption 17
5.2 Upper-Limb Kinematics and Center-of-Mass Trajectory Planning 18
5.2.1 *Geometric Analysis of Torso Dynamic Rotation* 18
5.2.2 *Local Trajectories of Both Arms and Inverse Kinematics* 18
5.3 Global Center-of-Mass Dynamics and Dynamic Compensation Model 19
6. Problem4: Model Construction and Solution **22**
6.1 Determination of Motor Power Based on the Lagrangian Energy Equation 22
6.2 Single-Objective Optimization Model. 23
6.3 Model Solution and Results Analysis 23
7. Model Evaluation **24**
7.1 Advantages and Limitations of the Model 24
7.2 Future Work 24
7.3 Conclusion 25
8. References. **25**
9. Appendix. **27**

I. Introduction

"To move with grace, machines must first learn to calculate with precision."

— Hans Moravec

Over the past few decades, the robotics industry has developed rapidly, emerging as one of the most dynamic and promising sectors in modern engineering and manufacturing.The development of the robotics industry is expected to reshape how work on modern assembly lines is organized and executed, while also providing a new and practical way to carry out tasks in difficult and complex environments. The study of robotic motion posture and the assurance that robots can operate safe and stable are of great importance. Such research not only improves the robot's operational reliability and motion precision but also provides a theoretical foundation and technical support for its long-term autonomous operation in complex environments.

Hangzhou Yushu Technology, a civilian robotics company, is preparing a technology-driven dance performance for a science and technology exhibition. To ensure that the show is both smooth and visually compelling, the company has commissioned us to develop models for the robots'dance postures and their motion stability.

1.1 Problem Restatement

Specifically, Yushu Technology requires that stability be evaluated from two perspectives: the safety of the robot motor, and the positioning of the robot's center of gravity. The former requires motor power and torque to remain within acceptable limits, while the latter requires the center of mass to stay inside the support area at all times. Data of the range of each robot joint's angles and the size of each part of the robot are supplied, which will help us build and solute the key models later.

To achieve our goals, specifically, we need to:

- Find the robot's leftmost position after extending and lifting its left hand forward at 60° and rotating it 30° left, and check that the motion is safe.
- Build a model to show the angles of a single leg as the robot moves from $(0, 0, 0)$ to $(10, 0, 0)$ at $2\,m/s$, and compute the time at which the knee joint exhibits the maximum change in its rotation angle during this motion.
- Build a model for the angles of the arms and legs during complex combined movements:The body rotates 45° to the left, while the arms trace 300-mm circles around the shoulders with a 4-second period, moving in opposite directions.
- Calculate the total energy the robot uses during the above movements and find a motion plan that completes the above actions with minimal energy use.

1.2 Literature Review

Early work has proposed several methods for robot motion planning and motor performance optimization problem. Liang et al. [1] and Fu et al. [2] conducted in-depth studies on kinematic calibration and joint-error compensation based on the D-H model, significantly improving the absolute positioning accuracy of robots. However, their work focuses mainly on static accuracy com-

pensation and does not address whole-body joint coordination or energy optimization in dynamic performance tasks. Chou et al. [3] proposed a trapezoidal-velocity synchronous planning method for energy-optimal joint trajectories, providing an effective scheme for efficiency optimization in joint space. Nevertheless, their model is designed for point-to-point motions of industrial manipulators and cannot handle the nonperiodic dynamic energy consumption of biped robots performing continuous and composite actions. Zeng et al. [4] introduced the pioneering ”leg-triangle” and ”arm-triangle” simplified models together with upper-body compensation theory, highlighting the importance of whole-body coordination for dynamic balance. Yet, their work centers on gait itself and does not incorporate motor-safety constraints or overall energy consumption into an integrated optimization framework.

In summary, existing research has made advances in improving local accuracy, optimizing specific energy-efficient trajectories, or maintaining gait stability, but the corresponding models are typically isolated and task-specific. They lack a unified framework capable of handling the complex performance tasks of mobile humanoid robots. The present study aims to integrate the strengths of these prior approaches by developing a comprehensive model that simultaneously considers whole-body dynamic coordination, real-time energy optimization, multi-joint synchronous motion, and motor-safety constraints. Unlike previous work, our model innovatively extends motion planning beyond isolated limbs or tasks and achieves integrated modeling and global optimization for the entire performance process of the Unitree G1 robot—from static poses to dynamic walking and further to complex dance motions.

1.3 Problem analysis

For Problem 1, we need to determine the end-effector position of the robot arm based on the robot’s parameters and joint angles. This requires specifying the transformation relationships between each link. To achieve this, we employ the standard D–H method to construct transformation matrices, which conveniently represent these relationships and allow us to obtain the end-effector coordinates.

For Problem 2, we aim to obtain the time-varying joint angles of the robot on stage. Based on human walking patterns, we divide a single step into two phases: double-support and single-support. Considering the robot’s actual motion, the center of mass (CoM) trajectory is described using a sinusoidal profile and the Linear Inverted Pendulum Model (LIPM). During the double-support phase, polynomial functions are used to model the velocity for smooth and continuous motion. In the single-support phase, the supporting leg is abstracted as an inverted pendulum and analyzed kinematically, while the swinging leg trajectory is modeled using a cycloid, as suggested in the literature. With the coordinate frames established via the D–H method in Problem 1, we can then compute each joint’s angles inversely. Given the high number of joints, a global random sampling combined with local fine search strategy is applied to accelerate the solution.

For Problem 3, the task is to determine the full-body joint trajectories while the robot performs a torso rotation combined with circular arm motions under double-support constraints. To address the challenges of time-varying reference frames caused by torso rotation and potential joint-limit violations in standard circular motions, we first establish a torso-following coordinate frame to decouple full-body motion. We then propose a workspace reconstruction and scapular rotation strategy to avoid mechanical dead points at the shoulder and satisfy physical joint limits, ensuring feasible mo-

tion. Considering that asymmetric composite motions can significantly shift the center of mass and destabilize the robot, we construct a multi-rigid-body dynamic model to synthesize the global CoM in real time. Using a quasi-static balance criterion and the inverted pendulum principle, we compute the required horizontal pelvis compensation. Finally, treating the lower limbs as a closed-chain mechanism, we apply geometric inverse kinematics based on the updated pelvis pose to precisely determine hip, knee, and ankle compensatory angles, achieving dynamic steady-state control of the whole body.

For Problem 4, we aim to quantify the robot's total energy consumption and perform trajectory optimization. Considering the nonlinear energy characteristics of humanoid robots—where static support dominates Joule (copper) losses, dynamic motion governs mechanical work, and energy recovery effects exist—we first establish a Lagrangian-based dynamic model to obtain instantaneous joint torques. Combining this with a DC motor equivalent circuit model, we formulate a comprehensive power equation that incorporates thermal loss coefficients and mechanical efficiency. For trajectory optimization, the infinite-dimensional functional extremum problem is converted into a finite-dimensional one by parameterizing joint trajectories with B-spline curves. Based on this, a nonlinear programming model is constructed under strict kinematic limits and ZMP stability constraints, and the optimal trajectory control points and timing parameters minimizing energy consumption are determined using a Sequential Quadratic Programming (SQP) algorithm.

1.4 Our Work

Our work can be illustrated as shown in Figure 1.

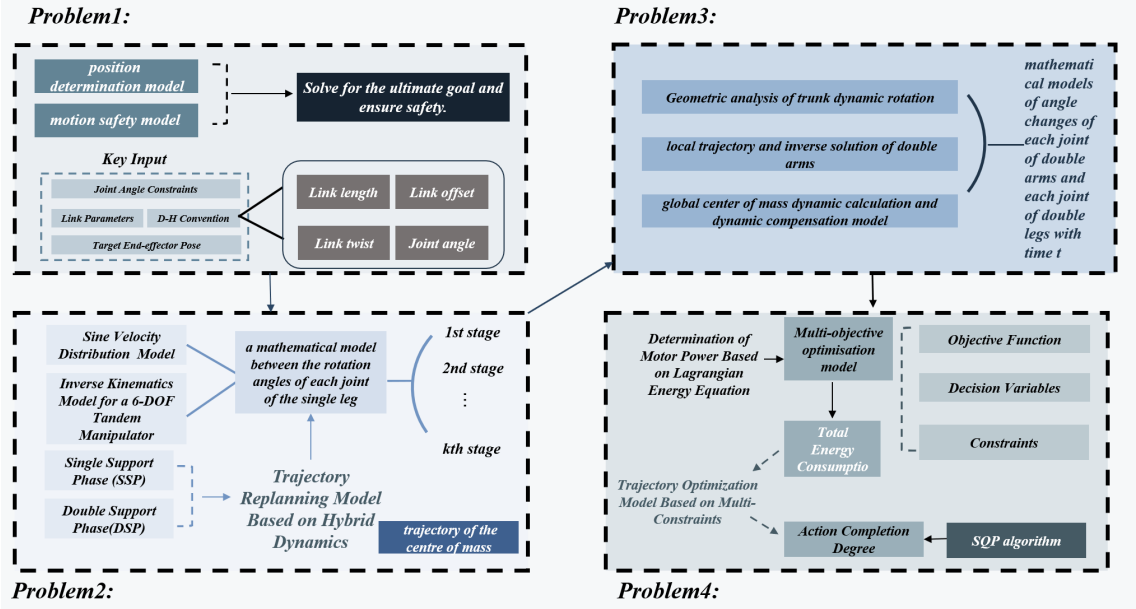


Figure 1 Our work

II. Assumptions and Notations

2.1 Assumptions

To simplify our model and eliminate the complexity, we make the following main assumptions in this literature.The following are the global assumptions; specific assumptions for each problem are introduced in their respective sections.

- **The robot's center of gravity coincides with its center of mass.** Since the robot's dimensions are negligible compared with the Earth's radius, the gravitational field around it can be regarded as uniform and parallel. In such a field, every mass element of the robot experiences the same gravitational acceleration, causing the resultant gravitational force to act at a single point in space that exactly matches the robot's center of mass.
- **To simplify the model, the movements of the upper body and lower limbs are decoupled, and passive torso sway is ignored.** Although lower-limb motion can cause some upper-body movement, high-performance humanoid robots can usually suppress small oscillations caused by walking, so the effects of minor upper-body sway can be neglected.
- **Each joint has a servo controller that follows its trajectory independently.** We acknowledge the physical coupling between joints and take it into account in the dynamics calculations; however, we assume that the low-level controllers can perfectly suppress the disturbances caused by this coupling, ensuring that each joint's actual motion matches the planned trajectory and simplifying model solution.
- **All robot links rigid with no deformation.** We ignore tiny elastic deformations and other minor effects, allowing us to focus on the robot's main geometric and dynamic behavior. This simplifies the complex continuum-mechanics problem into a solvable multi-rigid-body system and makes the later model construction more tractable.
- **Treat the upper and lower arms, as well as the upper and lower legs, as having equal lengths.** We acknowledge that the lengths of these robot segments may differ slightly in reality, but their influence on the global kinematics is negligible. To keep the model manageable, we idealize them as equal.

2.2 Notations

In this work, we use the nomenclature symbol in in the model construction. Other infrequently-used symbols will be introduced once they are used.

III. Problem 1: Model Construction and Solution

In this section, we propose the position determination and motion safety verify model. The position is determined using the Denavit-Hartenberg (D-H) method [6].Then in order to verify that the robot's dance movements meet safety requirements, we propose the motion safety model, considering motor safety and center-of-mass stability. During this process, we take into account nearly all influencing attributes from the given information. As a result, our model provides fairly accurate results for both position determination and motion safety evaluation.

Table 1 Notations

Symbol	Description	Unit
a_i	Link length, the distance along the Xaxis, the shortest distance from $Z_{i-1}$to Z_i	m
α_i	Link twist angle, the rotation angle about the X_i axis from Z_{i-1} to Z_i	°
d_i	Link offset, the distance along the Z_{i-1} axis, the offset from X_{i-1} to X_i	m
θ_i	Joint angle, the rotation angle about the Z_{i-1} axis from X_{i-1} to X_i	°
q_i	Rotation angle of the i-th joint	°
n	Number of joints to be modeled or the degrees of freedom of the system	
I_j	Moment of inertia matrix of the link	$kg{\cdot}m^2$
ω_j	Angular velocity vector of the link	rad/s
τ_i	Driving torque generated by the motor of the i-th joint	N • m
t_{to}	Total time of the robot's walking movement on the stage	s
t_{step}	Duration of a single gait cycle	s
t_{step}^{tr}	Duration of the single support phase (SSP) in a single gait cycle	s
H_{swing}	Maximum foot lifting height	m
M_{up}	Total mass of the robot's upper body	kg

3.1 Position Determination Model

According to [5] , the Unitree G1 robot arm possesses five degrees of freedom (DoFs), consisting of three DoFs at the shoulder and two at the elbow. Since the elbow rotation and the shoulder roll do not affect the robot's arm lifting and translation motion, only four of these DoFs need to be considered in our analysis. Following the official joint naming convention provided in[5] , the relevant joints are L_SHOULDER_PITCH, L_SHOULDER_ROLL, L_SHOULDER_YAW, and L_ELBOW, corresponding to the pitch, roll, and yaw motions of the shoulder and the pitch motion of the elbow, respectively.

The D-H method provides a simple and general way to model the links and joints of a robot, applicable to any robotic structure and to coordinate transformations in any reference frame[2]. Based on the D-H method, we establish the position determination model for the key node coordinates of the robot while performing each dance movement.

The procedure mainly consists of four steps: establishing the link coordinate frames, determining the D-H parameters, constructing the transformation matrices, and deriving the forward kinematic equations.

1. Establishing the link coordinate frames

To describe the kinematic model of the robot's left arm, we first define the global world coordinate system W : the origin is located at the center of the shoulder joint, X_W points toward the

front of the stage, Y_W points to the robot's left, and Z_W points upward.

Based on the physical structure of the Unitree G1 armand the D-H modeling rules[6] , we define the coordinate frame of the i-th link as i. The resulting local coordinate frames for all links are listed in the table below.

Table 2 Local Coordinate Frames

Coordinate Frame	Origin	x_i-axis Direction	y_i-axis Direction	z_i-axis Direction
Base Coordinate Frame	Shoulder joint center	Forward direction	Leftward direction	Upward direction
Yaw Joint Coordinate Frame	Coincides with $\{0\}$	Forward direction	Leftward direction	Upward direction
Pitch Joint Coordinate Frame	Coincides with $\{0\}$	Forward direction	Downward direction	Leftward direction
Elbow Joint Coordinate Frame	Elbow joint center	Downward direction	Backward direction	Parallel to Z_1

2. Determining the D-H parameters

For the transformation from frame $i-1$ to frame i, the D-H convention requires us specifying four parameters:

- **Link length** a_i: The distance along the X_i axis from Z_{i-1} to Z_i. This value is always non-negative.
- **Link twist** α_i: The angle about the X_i axis that rotates Z_{i-1} into Z_i, with the sign determined by the right-hand rule.
- **Link offset** d_i: The distance along the Z_{i-1} axis from X_{i-1} to X_i. For prismatic joints, this is the joint variable.
- **Joint angle** θ_i:The angle about the Z_{i-1} axis that rotates X_{i-1} into X_i. For revolute joints, this is the joint variable.

Each link contains exactly one joint variable: θ_i for a revolute joint and d_i for a prismatic joint.

3. Constructing the transformation matrices

We define $\mathbf{T_i^{i-1}}$ as the homogeneous transformation matrix that maps frame $i-1$ to frame i. The transformation matrix $\mathbf{T_i^{i-1}}$ is given by:

$$\mathbf{T_i^{i-1}} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

where θ_i、α_i and d_i denote joint angle、Link twist and link offset specially.

4. Deriving the forward kinematic equations

By multiplying all adjacent transformation matrices, we obtain the total transformation matrix $\mathbf{T}^0_{End}$, which gives the robot's left endpoint relative to the shoulder base.

$$\mathbf{T}^0_{End} = \mathbf{T}^0_1(\theta_1) \cdot \mathbf{T}^1_2(\theta_2) \cdot \mathbf{T}^2_3(\theta_3) \cdot \mathbf{T}^3_4(\theta_4) \cdot \mathbf{T}^4_5, \tag{2}$$

where $\mathbf{T}^{\mathbf{i-1}}_{\mathbf{i}}$ is the homogeneous transformation matrix from frame $i-1$ to frame i. The final endpoint position is given by the first three elements of the last column of T^0_{End}, which we denote as $\mathbf{P} = [x, y, z]^T$:

$$\mathbf{P} = \mathbf{T}^{\mathbf{0}}_{\mathbf{end}} \cdot \mathbf{A}, \tag{3}$$

where $\mathbf{A} = [0, 0, 0, 1]^T$, is a coefficient matrix.

3.2 Motion Safety Verify Model

When the robot performs this motion, the motor must operate within its safe torque limits. In this task, the required movement involves the joints L_SHOULDER_PITCH , L_SHOULDER_YAW and L_ELBOW. The robot arm consists of the upper arm and the forearm. Based on our assumptions, we approximate both parts as uniform rigid rods with the same length.

For the motor at the L_SHOULDER_PITCH joint, the output torque must balance the torque caused by gravity on the arm in the given posture. This balance ensures that the arm remains steady during the motion. In other words, the following condition must hold:

$$^{(shoulder)}\tau_{pitch} = ||\mathbf{r}_{ua} \times \mathbf{F}_{ua} + \mathbf{r}_{la} \times \mathbf{F}_{la}||, \tag{4}$$

where $^{(shoulder)}\tau_{pitch}$ denotes the output torque of the motor corresponding to the L_SHOULDER_PITCH joint. The vectors r_{ua} and r_{la} represent the directions from the origin of the global world frame to the centers of mass of the upper arm and lower arm, respectively. Since gravity acts vertically downward, opposite to the Z-axis, we have:

$$\mathbf{F}_{ua} = [0, 0, -m_{ua} * g]^T, \mathbf{F}_{la} = [0, 0, -m_{la} * g]^T, \tag{5}$$

where m_{ua} and m_{la} denote the masses of the upper and lower arm, respectively, and under our assumption, the two masses are identical.

For the torque of the L_SHOULDER_YAW joint, the gravity direction is always parallel to its rotation axis (the z-axis). Therefore, gravity produces no torque about the yaw axis. Thus, the torque $^{(shoulder)}\tau_{yaw}$ for this joint:

$$^{(shoulder)}\tau_{yaw} = 0. \tag{6}$$

Similarly, the torque expression for the L_ELBOW joint is given by:

$$^{(elbow)}\tau = ||\mathbf{r}^{'}_{la} \times \mathbf{F}_{la}||, \tag{7}$$

where $r^{'}_{la}$ is the position vector from the elbow joint to the center of mass of the lower arm, and it satisfies:

$$\mathbf{r}^{'}_{la} = \mathbf{r}_{la} - 2 * \mathbf{r}_{ua}, \tag{8}$$

where r_{la} is the position vector from the elbow joint to the center of mass of the lower arm.

Verify that the motors remain stable after the motion, i.e., confirm that $^{(shoulder)}\tau_{pitch}$, $^{(shoulder)}\tau_{yaw}$, and τ_{elbow} are all less than the motor's maximum torque τ_{limit}.

3.3 Model Results

Based on the figure below and geometric analysis, the D−H parameters are listed in the table below.

Table 3 D−H parameters

Link i	θ_i	d_i	a_{i-1}	α_{i-1}	**Physical Meaning**
1	30°	0	0	90°	Shoulder Pitch: Establishes the pitch plane
2	30°	0	0	0°	Shoulder Yaw: Establishes the yaw direction
3	0°	0	0.169	0°	Shoulder Roll: Establish the roll direction
4	0°	0	0.169	0°	Elbow (Upper Arm): Extends the upper arm
5	0°	0	0	0°	End Effector (Forearm): Extends the forearm

By substituting the parameters into (1), we obtain the following transformation matrices:

$$\mathbf{T_1^0} = \begin{bmatrix} -\sin\theta_1 & -\cos\theta_1 & 0 & 0 \\ \cos\theta_1 & -\sin\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{9}$$

$$\mathbf{T_2^1} = \begin{bmatrix} -\sin\theta_2 & 0 & \cos\theta_2 & 0 \\ \cos\theta_2 & 0 & \sin\theta_2 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{10}$$

$$\mathbf{T_3^2} = \begin{bmatrix} \cos\theta_3 & 0 & -\sin\theta_3 & L_1\cos\theta_3 \\ \sin\theta_3 & 0 & \cos\theta_3 & L_1\sin\theta_3 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{11}$$

$$\mathbf{T_4^3} = \begin{bmatrix} cos\theta_4 & 0 & -sin\theta_4 & L_2cos\theta_4 \\ sin\theta_4 & 0 & cos\theta_4 & L_2sin\theta_4 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{12}$$

$$\mathbf{T_5^4} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{13}$$

where θ_i is the joint angle associated with the corresponding joint, $L_i$denote the length of the corresponding link.

Using Eq.(2), we obtain the transformation matrix from the D−H base 0 to the end 5. The computed end-point coordinates are $(0.25350, 0.14636, 0.16900)(Unit : millimeters)$.

Notably, the motion specified in Problem 1 has a simple spatial geometry; the result from a geometric check agrees with our position-determination model, which further validates the model.

Furthermore, based on the robot parameters extracted from[5],we obtained the parameter values listed in the table below:

Table 4 Parameters of the Robot's Upper Limb

Parameters	m_{la}	m_{ua}	g	r_{la}	r_{ua}
Values	$1.394kg$	$1.394kg$	$9.8m/s^2$	$(253.5, 146.4, 169.0)^T$	$(63.38, 36.59, 42.25)^T$

*r components are measured in millimeters..

We analyze the geometric relations between joints and apply the motion safety model. The results show that the computed torques, ${}^{(shoulder)}\tau_{pitch} = 4.9986, \text{N} \bullet \text{m}$ and ${}^{(elbow)}\tau = 1.9994, \text{N} \bullet \text{m}$, both fall within the motor's safe operating limits $\tau_{limit} = 120N \cdot m$.The results confirm that the robot meets motor safety requirements while performing the arm-lifting and lateral-moving motion.

IV. Problem2: Model Construcion and Solution

In this section, we first introduce two key assumptions based on the robot's walking mechanics to facilitate subsequent model development. We then employ a sinusoidal velocity profile to characterize the robot's overall translational motion, and on this basis analyze the robot's behavior within a single gait cycle and construct the corresponding mechanistic model.

The robot's motion within a single gait cycle is divided into the single-support phase(SSP) and the double-support phase(DSP)[6]. Mechanistic models are established for each phase to describe the associated physical processes. After precisely formulating the robot's displacement, velocity, and acceleration at every moment throughout the gait cycle, we apply the inverse D－H method to obtain the joint angles of a single leg over time. Based on these results, we finally determine the moment at which the knee joint experiences the maximum angular variation during the entire motion.The overall model is shown in the figure below.

4.1 Global Assumptions

In particular, to facilitate the subsequent model derivation, we impose the following assumptions on the robot's walking motion:

Assume that the robot's center of mass remains at a constant height throughout the walking process. For a robot performing steady linear walking, the vertical fluctuation of the hip joint is typically minimal. We therefore approximate the center of mass height as constant throughout the walking process, which serves as the physical premise that enables the Linear Inverted Pendulum Model (LIPM) to admit an analytical solution[10] .

The supporting foot does not slip or rotate while it is in contact with the ground. The robot's soles are equipped with high-friction materials[5] . Under normal gait conditions, the static friction provided by the ground is sufficient to counteract horizontal forces acting on the feet, thereby effectively preventing slipping.

4.2 Sinusoidal-Profile-Based Global CoM Motion Planning Model

Problem 2 requires the robot to move from $(0, 0, 0)$ to $(10, 0, 0)$ along a straight line on the x-axis. During this motion, the y and z components of the robot's center of mass remain constant, so for the macroscopic motion we only consider the change along the x-axis, reducing the complex 3D problem to a 1D one.

First, for modeling the robot's walking speed, common approaches include trapezoidal velocity planning [3], sinusoidal velocity planning [7], polynomial velocity planning [8], and S-curve velocity planning [9].

However, polynomial velocity planning cannot simultaneously guarantee both monotonicity and continuity of the velocity curve when the robot's motion requires zero initial and final velocities with a fixed displacement. Under constraints such as prescribed average speed and total motion time, trapezoidal and S-curve velocity planning often have non-unique solutions, making it impossible to obtain exact analytical expressions for velocity or acceleration without introducing additional parameters.

In contrast, sinusoidal velocity planning naturally describes a smooth, continuous, and jerk-free non-uniform motion. It ensures continuous acceleration with no abrupt changes, enhancing motion safety. Therefore, we adopt the sinusoidal velocity planning model, using the following sinusoidal function to characterize the time-varying velocity:

$$v(t) = v_m \sin\left(\frac{\pi t}{t_{to}}\right), \tag{14}$$

where t_{to} is the total time of the robot's walk on the stage. From the problem data, $t_{to} = 5$s, which is constant.v_m is the maximum speed magnitude during the walk. $v(t)$ denotes the robot's speed at time t.

By integration, we can derive the relationship between the robot's maximum speed and its average speed:

$$v_{\text{avg}} = \frac{d}{t_{to}}, \tag{15}$$

where v_{avg} denotes the average speed, and d and t_{to} denote the robot's total displacement and total time, respectively, satisfying:

$$d = \int_0^{t_{to}} v_m \sin\left(\frac{\pi t}{t_{to}}\right) dt. \tag{16}$$

Based on the above velocity expression, we can derive formulas for the robot's center-of-mass displacement and acceleration at any given time:

$$S(t) = \int_0^t v(s)ds = \frac{v_m t_{to}}{\pi}\left(1 - \cos\left(\frac{\pi t}{t_{to}}\right)\right), \tag{17}$$

where $S(t)$ and $v(t)$ denote the robot's displacement and velocity at time t, respectively.

Furthermore, we can obtain the expression for the robot's acceleration at a given time t:

$$a(t) = \frac{dv}{dt} = \frac{\pi^2}{5}\cos\left(\frac{\pi t}{5}\right), \tag{18}$$

where $a(t)$ denotes the acceleration of the robot's center of mass at time t, and $v(t)$ represents the velocity of the center of mass at the same moment.

In summary, we establish the following global center-of-mass motion planning model based on a sinusoidal velocity profile:

$$\begin{cases} S(t) = \frac{v_m t_{to}}{\pi}\left(1 - \cos\left(\frac{\pi t}{t_{to}}\right)\right) \\ v(t) = v_m \sin\left(\frac{\pi t}{t_{to}}\right) \\ a(t) = \frac{x^2}{5}\cos(\frac{\pi t}{5}) \\ d = \int_0^{t_{to}} v_m \sin\left(\frac{\pi t}{t_{to}}\right) dt \end{cases}, \tag{19}$$

where t_{to} is the total walking time of the robot on the stage, which is obtained from the problem data and is constant, $t_0 = 5s$. v_m is the maximum speed during the robot's walk. $S(t)$,$v(t)$ and $a(t)$ denote the displacement, velocity, and acceleration of the robot at time t, respectively. d is the total displacement of the robot, which is constant, $d = 10m$.

4.3 Trajectory Replanning Model Based on Hybrid Dynamics

The robot's overall motion is described by the global sinusoidal velocity planning model. Within this framework, the robot's gait follows a fixed period, and its non-uniform motion is achieved by dynamically adjusting the stride length of each step. Specifically, the stride length increases during high-speed phases and decreases during low-speed phases, enabling continuous speed variation within a fixed time rhythm.

We first analyze the mechanistic model of the robot's walk within a single gait cycle. As mentioned in [10], within a single gait cycle, the robot's walking can be divided into a double-support phase (DSP) and a single-support phase (SSP), where the DSP accounts for 20% of the cycle and the SSP accounts for 80% of the cycle.For a single gait cycle (from time 0 to the cycle time t_{step}), the timeline can be divided as shown in the figure. During the k-th gait cycle of the robot, the time satisfies $t \in [(k-1)t_{step}, kt_{step}]$, where the time ranges for the SSP and DSP are $t \in [(k-1)t_{step}, (k-1)t_{step} + t_{step}^{tr}]$ and $t \in [(k-1)t_{step} + t_{step}^{tr}, kt_{step}]$, respectively.

The robot's motion phases within a single gait cycle over time are illustrated in the figure below.

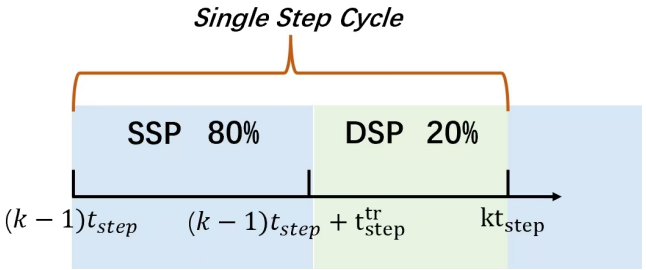


Figure 2 The robot's motion phases within a single gait cycle

4.3.1 *Single-Support Phase (SSP) Mechanism Model*

During the single-support phase (SSP) of bipedal robot walking, contact with the ground occurs only at the supporting foot, and the contact point of the supporting foot can be considered a fixed hinge for a short period. In this phase, the robot's overall dynamics are primarily determined by the pendulum-like swing of its center of mass around this contact point. Therefore, the robot can be

modeled as an inverted pendulum [10] (Inverted Pendulum Model). Specifically, the contact point between the supporting foot and the ground corresponds to the hinge of the pendulum, the robot's upper body and overall mass can be treated as a concentrated mass at the pendulum's end, and the supporting leg can be approximated as a massless rod, as illustrated in the figure . This inverted

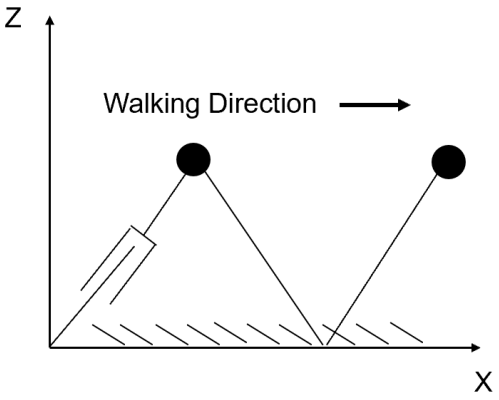


Figure 3 Inverted Pendulum Model

pendulum model preserves the key nonlinear instability characteristics of walking, providing both physical plausibility and mathematical tractability for describing center-of-mass motion, analyzing stability, and designing gait planning algorithms.

For the k-th gait cycle of the robot, we can obtain the following classic Linear Inverted Pendulum Model(LIPM) equation[10]:

$$\frac{\mathrm{d}^2 x(t)}{\mathrm{d}t^2} = \frac{g}{h}\left(x(t) - P_k\right), \tag{20}$$

where $t \in [(k-1)t_{step}, (k-1)t_{step}+t_{step}^{tr}]$, $x(t)$ represents the horizontal displacement of the robot's center of mass over a single gait cycle as a function of time, g is the gravitational acceleration, taken as $g = 9.8\, m/s^2$, h is the constant height of the center of mass relative to the support point, and P_k denotes the position of the foot support point during the k-th gait cycle.

Within a single gait cycle, the durations of the foot lift and landing phases are symmetric, so the position of the support foot corresponds to the midpoint coordinate of this step. Thus, the midpoint time of the k-th step can be expressed as:

$$t_k^{mid} = (k-1)t_{step} + 0.5t_{step}, \tag{21}$$

where $t^{mid}k$ denotes the midpoint time of the k-th step, and $tstep$ is the duration of a single step. The position of the support foot during the single-support phase of the k-th gait cycle is then given by:

$$P_k = S(t_k^{mid}), \tag{22}$$

where $t^{mid}k$ is the midpoint time of the k-th step, $Tstep$ is the duration of a single step, and P_k denotes the position of the robot during the single-support phase of the k-th step.

We apply the Laplace transform to the Eq.(20) and solve it algebraically to obtain the inverted pendulum dynamics, which describe the robot's center-of-mass trajectory as well as its velocity and acceleration relative to the world coordinate system.

For each single-support phase period $t \in [(k-1)t_{step}, (k-1)t_{step} + t_{step}^{tr}]$, we have:

$$\begin{cases} x_{\text{ssp}}(t) = (x_0 - P_k)\cosh\left(\dfrac{t}{t_{to}}\right) + t_{to}v_0 \sinh\left(\dfrac{t}{t_{to}}\right) + P_k \\ v_{\text{ssp}}(t) = \dfrac{x_0 - P_k}{t_{to}} \sinh\left(\dfrac{t}{t_{to}}\right) + v_0 \cosh\left(\dfrac{t}{t_{to}}\right) \\ a_{\text{ssp}}(t) = \dfrac{x_0 - P_k}{t_{to}^2} \cosh\left(\dfrac{t}{t_{to}}\right) + \dfrac{v_0}{t_{to}} \sinh\left(\dfrac{t}{t_{to}}\right) \end{cases}, \tag{23}$$

where $t_c = \sqrt{h/g}$,determines the system's instability and the speed of the center-of-mass oscillation [10]. x_k^{ssp} is the position of the robot's center of mass in the world coordinate system during the k-th gait cycle, and $v^{ssp}k$, $a^{ssp}k$ are its velocity and acceleration, respectively. $xk, ssp^{(0)}$ and $vk, ssp^{(0)}$ are the displacement and velocity at the beginning of the k-th gait cycle, satisfying:

$$\begin{cases} x_{k,ssp}^{(0)} = S((k-1)t_{step} \\ v_{k,ssp}^{(0)} = v((k-1)t_{step} \end{cases}, \tag{24}$$

where the definitions of $S((k-1)t_{step})$ and $v((k-1)t_{step})$ are given in Eq: (19).

4.3.2 *Double-Support Phase Mechanism Model*

Within each gait cycle, the end-state of the LIMP-based trajectory may deviate from the global sinusoidal velocity plan. To avoid sudden changes during step transitions, we use a fifth-order polynomial to describe the Double Support Phase (DSP) motion. By leveraging the boundary constraints of the fifth-order polynomial, we fit the start and end points of the Single Support Phase (SSP) between consecutive gait cycles. This approach ensures continuous acceleration, eliminates rigid impacts, and provides a closed-loop effect, keeping the robot consistently following the global sinusoidal velocity trajectory without divergence.

For the k-th gait cycle, we construct the following fifth-order polynomial for the center-of-mass displacement:

$$x_k^{dsp}(t) = \sum_{i=0}^{5} c_i t, \tag{25}$$

the corresponding velocity and acceleration expressions are:

$$\begin{cases} v_k^{dsp}(t) = \frac{dx_k^{dsp}}{dt} \\ a_k^{dsp}(t) = \frac{d^2 x_k^{dsp}}{dt^2} \end{cases}, \tag{26}$$

where $t \in [(k-1)t_{step} + t_{step}^{tr}, kt_{step}]$, $x_k(t)$ is the center-of-mass trajectory, and c_i are the parameters to be fitted.

This fifth-order polynomial has six boundary conditions, including three constraints at the start point and three constraints at the end point:

Startpoint constraints: At the end of the SSP within the same gait cycle, the centroid displacement must equal the displacement at the start of the DSP. This ensures that the trajectory is twice continuously differentiable and consistent with the robot's actual walking, i.e., it must satisfy:

$$\begin{cases} x_k^{dsp}((k-1)t_{step} + t_{step}^{tr}) = x_k^{ssp}((k-1)t_{step} + t_{step}^{tr}) \\ v_k^{dsp}(k-1)t_{step} + t_{step}^{tr}) = v_k^{ssp}((k-1)t_{step} + t_{step}^{tr}) \\ a_k^{dsp}((k-1)t_{step} + t_{step}^{tr}) = a_k^{ssp}((k-1)t_{step} + t_{step}^{tr}) \end{cases}, \tag{27}$$

where x_k^{dsp}, v_k^{dsp}, and a_k^{dsp} represent the displacement, velocity, and acceleration at the start of the DSP phase in the kth gait cycle, while x_k^{ssp}, v_k^{ssp}, and a_k^{ssp} are defined in Eq:(**??**).

Endpoint constraints: During walking, each step follows the global sinusoidal velocity plan. Since our mechanistic analysis is based on a single gait cycle under this global planning, at the end of each gait cycle, the robot's state should return to the globally planned state, which requires:

$$\begin{cases} x_k^{dsp}(kt_{step}) = S(kt_{step}) \\ v_k^{dsp}(kt_{step}) = \frac{dS(kt_{step})}{dt} \\ a_k^{dsp}(kt_{step}) = \frac{d^2S(kt_{step})}{dt} \end{cases}, \tag{28}$$

where x_k^{dsp},v_k^{dsp},a_k^{dsp} correspond to the robot's center-of-mass displacement, velocity, and acceleration during the DSP phase, as referenced in (25) and Eq. (26). S denotes the displacement under the global planning, see Eq. (17).

4.4 6-DoF Serial Manipulator Inverse Kinematics Model

4.4.1 *D-H Method Construction*

Although a 6-DoF robot usually uses a decoupled position-and-orientation solution, in this linear walking scenario the primary motion occurs in the sagittal plane, mainly contributed by the three pitch DOFs at the hip, knee, and ankle. Therefore, we can set $\theta_1 = \theta_2 = \theta_6 = 0°$, to simplify the problem to a "planar 3-link inverse solution." However, to maintain generality and rigor, we construct the full 6-DOF D-H model.

The D-H kinematic chain follows the sequence: base, hip yaw, hip abduction, hip pitch, knee, ankle pitch, ankle roll, and foot end. Similarly, we define the four key parameters—link length (a_i), link twist (α_i), link offset (d_i), and joint angle (θ_i)—following the kinematic chain order. The origins of the base frame and the three hip frames are all located at the hip joint, while the origins of the knee and ankle frames are located at the knee and ankle joints, respectively.

The transformation matrix $\mathbf{T}_i^{i-1}$ is defined as in Eq. (1). The complete forward kinematics from the hip joint to the foot end is given by:

$$T_{Foot}^{Hip} = T_1^0 \cdot T_2^1 \cdot T_3^2 \cdot T_4^3 \cdot T_5^4 \cdot T_6^5. \tag{29}$$

Furthermore, using the forward kinematics model, we can obtain the position vector of the foot end relative to the hip base coordinate system, **P**, which corresponds to the last column of the transformation matrix $\mathbf{T}_{\mathbf{Foot}}^{\mathbf{Hip}}$:

$$\mathbf{P} = \mathbf{T}_{Foot}^{Hip} \cdot \mathbf{A}. \tag{30}$$

4.4.2 *Inverse Kinematics*

The inverse kinematics problem is defined as follows: given the position of the end-effector relative to the hip base coordinate system at time t, solve for the joint angles θ_i $(i = 1, 2, ..., 6)$[6].

To achieve a stable gait, the swing leg's foot trajectory needs to be planned. Therefore, the input for the inverse kinematics solution is the target position of the foot relative to the hip joint

base coordinate system:

$$\mathbf{P}_{\text{target}}(t) = \begin{bmatrix} x_{\text{foot}}(t) - x_{\text{LIPM}}(t) \\ 0 \\ z_{\text{foot}}(t) - H_{\text{hip}} \end{bmatrix}, \tag{31}$$

where H_{hip} is the initial height of the hip joint, which is a constant $H = 0.48, \text{m}$, and the position vector of the ankle joint in the world coordinate system is $[x_{\text{foot}}(t), y_{\text{foot}}(t), z_{\text{foot}}(t)]^T$.

The height coordinate of the ankle joint in the world coordinate system (i.e., the z-axis component) is described using a cycloid equation. The cycloid equation represents the trajectory of a fixed point on the circumference of a circle rolling along a straight line, given in parametric form as:

$$\begin{cases} x = R(t - \sin t) \\ y = R(1 - \cos t) \end{cases}, \tag{32}$$

where R is the radius, and t represents the time of motion.

To meet the robot's gait requirements, map the standard cycloid equation parameters to establish a time–angle relationship:

$$t = \frac{2\pi \cdot \tau}{t_{step}^{tr}}, \tag{33}$$

where t_{step}^{tr} represents the duration of the single-leg support phase within one gait cycle. The corresponding parameter R can be expressed by the following formula:

$$2R = H_{swing}, \tag{34}$$

where H_{swing} represents the maximum foot-lift height.

Substitute the above parameters into the cycloid parametric equations (32). We can then obtain the time-varying position of the robot's ankle as follows:

$$\begin{cases} x_{foot}(\tau) = \frac{H_{swing}}{2}\left(\frac{2\pi\cdot\tau}{t_{step}^{tr}} - \sin\left(\frac{2\pi\cdot\tau}{t_{step}^{tr}}\right)\right) \\ z_{foot}(\tau) = \frac{H_{swing}}{2}\left(1 - \cos\left(\frac{2\pi\cdot\tau}{t_{step}^{tr}}\right)\right) \end{cases}, \tag{35}$$

where substitute this coordinate $\mathbf{P} = (x_{foot}, 0, z_{foot})^T$ into the above equations, and use matrix operations to solve for the equation's parameters.

4.5 Model Solution

First, we calculate the center of mass displacement based on the sinusoidal-profile-based global CoM motion planning model and the trajectory replanning model based on hybrid dynamics.The solution approach is illustrated in the figure below.

Then, based on the obtained CoM displacement, velocity, and acceleration at each moment, we solve the 6-DoF serial manipulator inverse kinematics model to obtain the joint angles of the single leg at a given time. The core objective is to solve (29), that is, to determine the joint angles contained in $\mathbf{T}_{Foot}^{Hip}$.

For this nonlinear inverse kinematics problem, we need to determine the unknown parameters in the transformation matrix $\mathbf{T}^{Hip}Foot$ based on the CoM displacement obtained from the motion planning described above. By minimizing the distance between the end-effector position computed from $\mathbf{T}^{Hip}Foot$ and the actual foot position from the planned trajectory, we convert the problem

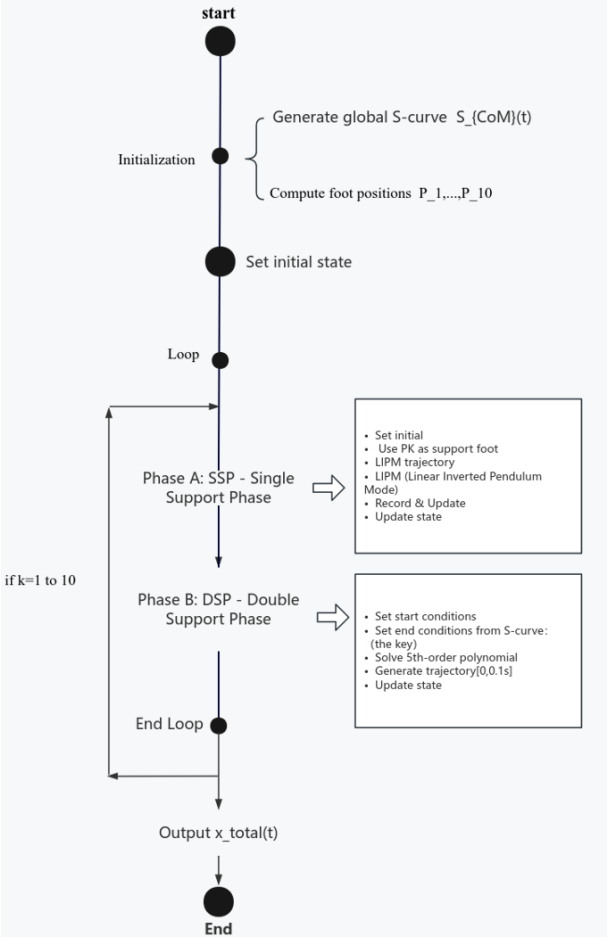


Figure 4 Modeling Approach

into an optimization task, seeking the joint angle parameters that make the foot end position as close as possible to the desired trajectory.

Because the number of parameters to optimize is large, the search space is extensive. We adopt a “global sampling + local search” strategy to find the optimal parameters. A global optimization algorithm is first used to enhance solution robustness, with GlobalSearch generating 500 initial points for exploration. Then, a Sequential Quadratic Programming (SQP) algorithm is applied for fine local search, transforming the original problem into multiple quadratic programming subproblems that are solved iteratively. The iteration parameters are set as follows: maximum iterations = 1000, function evaluations = 3000, step tolerance = 10^{-10}, and optimality tolerance = 10^{-10}.

Finally, the time at which the knee joint undergoes its maximum angular change is $1.77\ s$, with a corresponding maximum knee angle of $73.7397°$. The variation of each single-leg joint angle during the robot’s overall motion is illustrated in the figure below:

From the above results, it can be seen that the angles of all joints in the robot’s left leg remain within the mechanical limits throughout the 5-second walking period, indicating that the motion design is safe and reasonable. Specifically, the hip joint’s pitch and yaw angles exhibit periodic changes, reflecting the characteristic leg lifting and swinging of a gait cycle. The hip roll and knee joint angles also show regular periodic flexion and extension, ensuring sufficient step length and support. The ankle pitch and roll angles function to adjust balance, with small and smooth variations

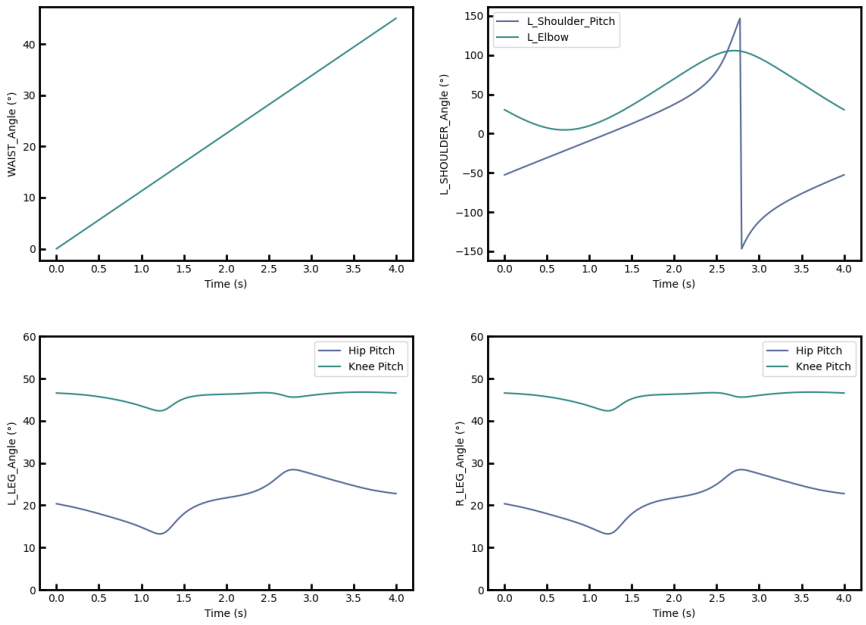


Figure 5 Time-Varying Trajectories of Key Joint Coordinates

that align with the physical requirements for gait stability. Overall, the trends of the joint angles are consistent with typical robotic walking patterns, and the results are both reasonable and physically intuitive.

V. Problem3: Model Construcion and Solution

In this section, we study how a humanoid robot can maintain full-body dynamic balance while performing a specified upper-limb motion (both arms drawing circles). The motion of the arms and torso inevitably causes continuous shifts in the system's center of mass. To keep both feet stationary and ensure that the projection of the full-body center of mass always remains within the support polygon, the lower body must execute precise, dynamic compensatory movements.

To this end, we established a "top-down, inverse compensation" modeling framework, which includes four processes: upper-limb dynamics modeling, global center-of-mass trajectory calculation, pelvic balance compensation, and lower-limb inverse kinematics solving.

5.1 Model Assumption

1.During the Yushu robot's body and arm rotation, the motion is assumed to be uniform. Since the problem does not provide constraints on motor acceleration or deceleration, we lack the basis to model non-uniform motion phases accurately. Under these circumstances, assuming uniform motion is the only feasible and reasonable simplification. This allows us to focus on analyzing the core motion trajectory and spatial geometric relationships, enabling the determination of a definite solution.

2.The circular trajectories of both arms are strictly planar. The problem explicitly provides the circle radius and period, without information on random disturbances or tracking errors. Therefore, we idealize the circular motion to simplify our model.

5.2 Upper-Limb Kinematics and Center-of-Mass Trajectory Planning

5.2.1 *Geometric Analysis of Torso Dynamic Rotation*

To better define the rotation angles and the positions of each joint, two coordinate systems are established: a torso-fixed coordinate system and a world coordinate system. Both systems take the center of the robot' s waist as the origin, with the x-axis pointing forward, the y-axis pointing to the left, and the z-axis pointing upward.

It is worth noting that the world coordinate system remains fixed, while the torso-fixed coordinate system rotates around the origin as the robot moves. The torso rotates uniformly around the waist (with the z-axis as the rotation axis), which allows us to derive:

$$\theta_{waist}(t) = \omega_{waist} t, \tag{36}$$

where w_{waist} is the angular velocity of the waist rotation. In the context of this study, it is a constant, with $\omega_{\text{waist}} = \pi/16$ rad/s.

Furthermore, we can obtain the rotation matrix of each key joint from the torso-fixed coordinate system to the world coordinate system at a given time t:

$$\mathbf{R}_{trunk}(t) = \begin{bmatrix} \cos(\theta_{waist}(t)) & -\sin(\theta_{waist}(t)) & 0 \\ \sin(\theta_{waist}(t)) & \cos(\theta_{waist}(t)) & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{37}$$

where $\mathbf{R}_{\text{trunk}}$ is the rotation matrix, and θ_{waist} is the rotation angle.

5.2.2 *Local Trajectories of Both Arms and Inverse Kinematics*

In the torso-fixed coordinate system, both arms still perform standard circular motions, with opposite directions. Suppose the left arm moves counterclockwise; then the right arm moves clockwise. Due to joint angle limits, to achieve the ideal circular motion assumed, the plane of the arm circles rotates around the shoulder joints, and the circle centers are no longer located at the shoulders. They are translated forward by δx and upward by δz, keeping the motion within a naturally reachable space, which aligns with humanoid kinematic characteristics. Taking the natural downward position of both arms as the initial state, we first consider the offset of the circle centers. Using kinematic formulas, we obtain:

$$\begin{cases} x_L(t) = R \cdot \sin(\frac{2\pi}{T} t) + \delta x \\ y_L(t) = d_y \\ z_L(t) = -R \cdot \cos(\frac{2\pi}{T} t) + \delta z \end{cases}, \tag{38}$$

$$\begin{cases} x_R(t) = -R \cdot \sin(\frac{2\pi}{T} t) \\ y_R(t) = -d_y \\ z_R(t) = -R \cdot \cos(\frac{2\pi}{T} t) \end{cases}, \tag{39}$$

where $\mathbf{P}_{L,raw}^{local}(t) = [x_L, y_l, z_l]^T$ and $\mathbf{P}_{R,raw}^{local} = [x_R, y_R, z_R]^T$ represent the positions of the left and right wrist centers in the torso-fixed coordinate system. Here, R is the radius of the arm's circular motion, d_y is the lateral intrinsic offset of the hand (a constant), and δx and δz are the offsets of the circle center along the x-axis and z-axis, respectively. Furthermore, to ensure humanoid-like motion, considering rotation in the scapular plane, the final position coordinates are:

$$\mathbf{P}_L^{local}(t) = \mathbf{R}_{scap} \cdot \mathbf{P}_{raw}^L(t)\mathbf{P}_R^{local}(t) = \mathbf{R}_{scap} \cdot \mathbf{P}_{raw}^R(t), \tag{40}$$

where $\mathbf{R}_{\text{scap}}$ is the rotation matrix of the scapular plane.

The elbow and shoulder joints are defined relative to the torso. Their angle computation depends only on the local trajectories and is independent of the rotation of the torso. Therefore, we can directly apply the law of cosines to obtain the shoulder and elbow joint angles. Taking the left arm as an example, we have:

$$\begin{cases} \theta_{elbow} = \pi - \arccos\min\left(\max\left(\frac{L_1^2+L_2^2-L_{reach}^2}{2L_1L_2}, -1\right), 1\right) \\ \theta_{pitch} = \arctan(z, x) \\ \theta_{yaw} = \arctan(x, \text{sign} \cdot R) \end{cases}, \tag{41}$$

where θ_{elbow} and θ_{pitch}, θ_{yaw} are the rotation angles of the elbow joint and the shoulder joint, respectively. L_1 and L_2 denote the lengths of the upper arm and forearm. ψ is a constant geometric correction term caused by elbow flexion. L_{reach} is the straight-line distance between the base and the end of the robotic arm:

$$L_{reach}^{left}(t) = \sqrt{x_L(t)^2 + d_y^2 + z_L(t)^2} \tag{42}$$

Similarly, the corresponding joint angle expressions for the right arm can also be obtained.

5.3 Global Center-of-Mass Dynamics and Dynamic Compensation Model

1.Center-of-Mass Dynamics Calculation

Unlike the previous model, the variation of the center of mass results from both the arm rotation and the torso rotation. Therefore, the analysis must be conducted in the world coordinate system. We can obtain:

$$\mathbf{C}_i^{world}(t) = \mathbf{R}_{trunk}(t) \cdot \mathbf{C}_i^{local}(t), \tag{43}$$

where $\mathbf{C}i^{world}$ and $\mathbf{C}i^{local}(t)$ are the position vectors of the local center of mass of segment i in the world coordinate system and the torso-fixed coordinate system, respectively. $\mathbf{R}_{trunk}(t)$ is the rotation matrix.

[4] provides the formula for calculating the center of mass of a biped robot:

$$\mathbf{C}_{up}^{world}(t) = \frac{\sum m_i \mathbf{C}_i^{world}(t)}{M_{up}}, \tag{44}$$

where M_{up} is the total mass of the robot's upper body, and $\mathbf{C}_i^{world}$ is the position vector of the i-th upper-limb segment in the world coordinate system. m_i denotes the mass of segment i

2.Pelvic Orientation Tracking [4] states that, to prevent a biped robot from tipping over or falling during motion, the projection of its upper-body center of mass on the support plane must remain at the midpoint of the two feet, that is, at the origin. In Problem 3, the robot is required

to maintain balance while performing complex combined motions by appropriately adjusting the joint angles of both legs. Therefore, during upper-body motion, the pelvis must generate a real-time compensatory movement to counteract the moment induced by the upper-body dynamics [**?**].

We can obtain:

$$\begin{cases} x_{pelvis}(t) = -\frac{M_{up}}{M_{low}} \cdot x_{C_{up}}^{world}(t) \\ y_{pelvis}(t) = -\frac{M_{up}}{M_{low}} \cdot y_{C_{up}}^{world}(t) \end{cases}, \tag{45}$$

where x_{pelvis} and y_{pelvis} represent the movement directions of the pelvis. The terms $x_{C_{up}}^{world}(t)$ and $y_{C_{up}}^{world}(t)$ denote the x- and y-components of the upper-body center of mass in the world coordinate system. M_{up} and M_{low} are the total masses of the robot's upper body and lower body, respectively.

Lower-Limb Closed-Chain Inverse Kinematics

To keep both feet fixed (the support area remains stationary) while generating the compensatory pelvis displacement $\mathbf{P}_{pelvis}(t) = [x_p(t), y_p(t), z_p]^T$ (where $z_p = Hwaist$ is the pelvis height), we must solve for the joint angles of both legs. The solution must ensure that the end positions of the two feet remain identical to their initial fixed locations, and that the foot soles stay parallel to the ground.

During motion, the robot's feet remain fixed while the pelvis translates to $(x_{pelvis}, y_{pelvis}, H_{waist})$, with its orientation unchanged. For simplicity, we take the left leg as an example (the right leg is symmetric) to calculate the joint flexion angles for a given pelvis position.

The left hip joint is located at W_{hip} on the left side of the pelvis center:

$$\mathbf{P}_{hip}(t) = [x_p(t),\ y_p(t) + W_{hip},\ z_p(t)]^T, \tag{46}$$

where $\mathbf{P}_{hip}$ denotes the position of the hip joint center in the world coordinate system, and $Whip$ is the lateral offset of the hip joint from the body center.We define the vector from the hip joint to the center of the foot sole as a virtual leg vector:

$$\mathbf{V} = \mathbf{P}_{foot} - \mathbf{P}_{hip}(t) = [v_x,\ v_y,\ v_z]^T, \tag{47}$$

where $\mathbf{V}$ is the virtual vector pointing from the hip joint to the foot center. $\mathbf{P}foot$ and $\mathbf{P}hip$ are the positions of the foot center and hip joint center in the world coordinate system, respectively, with the following components:

$$\begin{cases} v_x = -x_p(t) \\ v_y = W_{step} - (y_p(t) + W_{hip}) \\ v_z = -z_p(t) \end{cases} \tag{48}$$

In the coronal plane (Y-Z) projection, the lateral swing of the leg is jointly performed by the hip roll and ankle roll. We first calculate an approximate hip roll angle, assuming that the ankle roll absorbs the remaining angle:

$$\theta_{roll} = \arctan\left(\frac{v_y}{-v_z}\right), \tag{49}$$

where v_y is the lateral offset, and v_z is the vertical height of the hip joint above the ground.

As the leg moves laterally, we need to calculate the equivalent projected length of the leg in the sagittal plane to facilitate the subsequent calculation of knee flexion:

$$L_{sagittal} = \sqrt{v_y^2 + v_z^2}, \tag{50}$$

where $L_{sagittal}$ is the length of the leg vector projected onto the sagittal plane.

In the sagittal plane, the virtual leg length is:

$$L_{virtual} = \sqrt{v_x^2 + L_{sagittal}^2}, \tag{51}$$

where $L_{sagittal}$ is the length of the leg vector in the sagittal plane, and v_x is the x-component of the leg vector. The knee joint angle can then be obtained using the law of cosines:

$$\theta_{knee} = \pi - \arccos\min\left(\max\left(\frac{L_3^2 + L_4^2 - L_{virtual}^2}{2L_3L_4}, -1\right), 1\right), \tag{52}$$

where θ_{knee} is the knee flexion angle, and L_3 and L_4 are the lengths of the thigh and shank, respectively.Similarly, the hip joint angles can be determined as:

$$\theta_{hip,pitch} = \alpha + \beta, \tag{53}$$

where $\alpha = \arctan\left(\frac{-v_x}{L_{sagittal}}\right)$, and $\beta = \arccos\min\left(\max\left(\frac{L_3^2+L_{\text{virtual}}^2-L_4^2}{2L_3L_{\text{virtual}}}, -1\right), 1\right)$.

Under the assumption that the foot sole remains parallel to the ground, the sum of the hip, knee, and ankle pitch angles should equal the torso pitch angle. Therefore, the ankle joint angle must satisfy:

$$\theta_{ankle,pitch} = -(\theta_{hip,pitch} + \theta_{knee}), \tag{54}$$

where $\theta_{ankle,pitch}$, $\theta_{hip,pitch}$, and θ_{knee} are the pitch angles of the ankle, hip, and knee joints, respectively.

Finally, the time-varying profiles of the key joint angles are shown in the figure below.

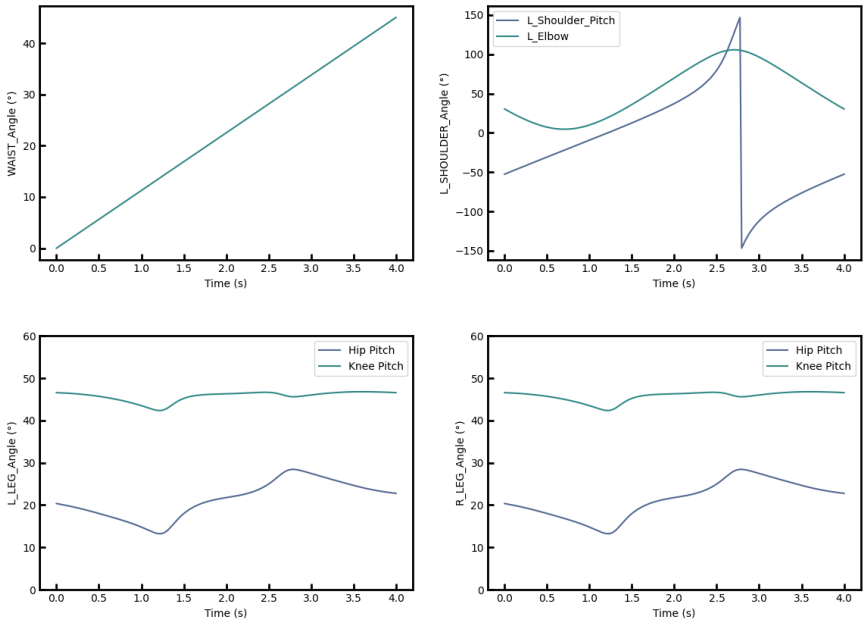


Figure 6 Joint Angel with Time

VI. Problem4: Model Construction and Solution

6.1 Determination of Motor Power Based on the Lagrangian Energy Equation

For the robot dance motion, we take the joint angles as generalized coordinates:

$$\mathbf{q}(t) = \begin{bmatrix} q_1(t) \\ q_2(t) \\ \cdots \\ q_n(t) \end{bmatrix}, \tag{55}$$

where q_i = rotation angle of the i-th joint, n = number of joints to be modeled, i.e., the system's degrees of freedom.

Physically, $\frac{d\mathbf{q}}{dt}$ and $\frac{d^2\mathbf{q}}{dt^2}$ represent the vectors of joint angular velocities and angular accelerations, respectively.

According to the Lagrangian dynamics principle [4], the motion equation of the Yushu humanoid robot is:

$$M(\mathbf{q})\frac{\mathrm{d}^2\mathbf{q}}{\mathrm{d}t^2} + C\left(\mathbf{q}, \frac{\mathrm{d}\mathbf{q}}{\mathrm{d}t}\right)\frac{\mathrm{d}\mathbf{q}}{\mathrm{d}t} + G(\mathbf{q}) = \boldsymbol{\tau}(t), \tag{56}$$

where $\mathbf{q}$ denotes the joint angles, and $M(\mathbf{q})$ is the inertia matrix, which is a symmetric positive-definite matrix.$C\left(\mathbf{q}, \frac{\mathrm{d}\mathbf{q}}{\mathrm{d}t}\right)$ represents the Coriolis and centrifugal torque matrix. $G(\mathbf{q})$ represents the Coriolis and centrifugal force matrix. $\boldsymbol{\tau}(t)$ is the vector of joint torques.

Generalized inertia matrix$\mathbf{M}(\mathbf{q})$

For the i-th joint of the robot, we assume that all subsequent joints are locked and treat the robot as a composite rigid body. The generalized inertia matrix can then be obtained by calculating the moment of inertia of this composite body about the current joint axis.

Coriolis and centrifugal terms, C(q

Centrifugal force is the outward force generated by a single joint rotating at high speed, while Coriolis force refers to the coupling force produced when one joint rotates while another joint moves. They satisfy:

$$C_{ijk} = \frac{1}{2}\left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i}\right), \tag{57}$$

where M_{ij} is an element of the inertia matrix, and q is the rotation angle.

Gravity term, G(q)

The gravity term represents the torque that the motors must generate to maintain the robot's current static posture against gravity. It satisfies:

$$G_i(\mathbf{q}) = \frac{\partial V(\mathbf{q})}{\partial q_i}, \tag{58}$$

where $V(\mathbf{q})$ is the total gravitational potential energy of all links, given by $V = \sum m_i g h_i(\mathbf{q})$.

Furthermore, using the motor torque as a bridge, based on the mapping between robot power loss, motor torque, and motor angular velocity established in [11], we have:

$$P_i(t) = \tau(t) \cdot \frac{dq_i}{dt}, \tag{59}$$

where P_{elec} is the motor output power, τ is the corresponding motor torque, and q represents the joint angles. The total energy E_{total} consumed by all n joints of the Yushu robot over the entire time

period T is:

$$E_{total} = \int_0^T \sum_{i=1}^{n} P_i(t)\, dt \tag{60}$$

6.2 Single-Objective Optimization Model

1.Objective Function

The objective is to minimize the total energy consumed by the robot to perform the dance motion:

$$\min_{\mathbf{X}} E_{total}(\mathbf{X}) \tag{61}$$

2.Constraints

Joint Kinematic Constraints

The robot's joints should not move excessively; each joint must satisfy its own angular limit constraints:

$$|\dot{q}_i(t)| \leq V_{\text{max}} \tag{62}$$

$$|\ddot{q}_i(t)| \leq A_{\text{max}} \tag{63}$$

$$q_{\text{min}} \leq q_i(t) \leq q_{\text{max}} \tag{64}$$

Motor Torque Constraints

According to the motor operating conditions described in [5], the torque of each motor must not exceed its peak value during operation, i.e., it must satisfy:

$$|\tau_i(t)| \leq \tau_{peak} \tag{65}$$

Voltage Constraints

The motors must not exceed their maximum operating voltage during operation.

$$|K_e \cdot \dot{q}_i(t)| \leq U_{max} \tag{66}$$

Stability Constraints

During walking, the projection of the robot's center of mass must remain within the support region.

$$P_{ZMP}(t) \in S_{support} \tag{67}$$

Performance Effect Constraints

In a specific stage performance context, the robot's motions must meet the performance requirements, including constraints on the radii of the three circular trajectories.

$$||\mathbf{P}_{hand} - \mathbf{P}_{shoulder}|| = 0.3 \tag{68}$$

6.3 Model Solution and Results Analysis

Using a time step of $dt = 0.01$, s, the robot's coordinates and their first- and second-order derivatives were generated as trajectory curves. The solution approach follows that of Problem 2, employing the SQP algorithm for optimization. Before optimization, the robot consumed approximately 77.46J of energy to perform the dance motion; after optimization, the total energy consumption was reduced by an average of 15.68%.

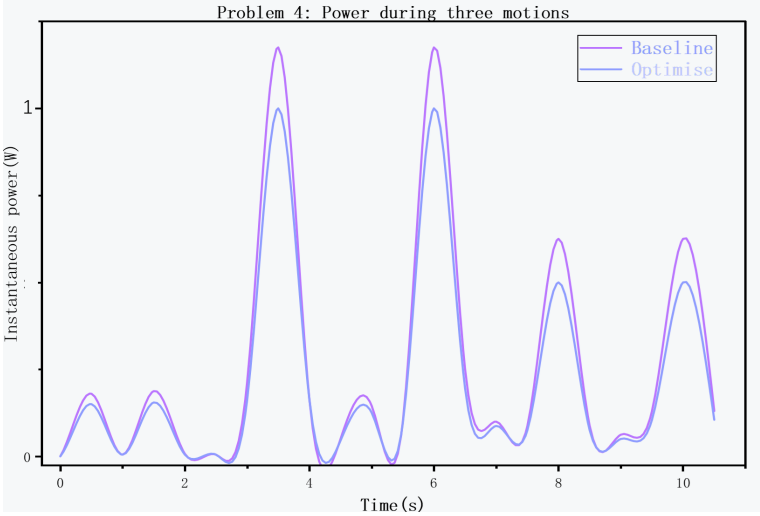


Figure 7 Comparison Before and After Optimization

VII. Model Evaluation

7.1 Advantages and Limitations of the Model

1.Advantages

Close to reality, logically rigorous, and highly interpretable. In the modeling process, we extensively used mechanistic analysis and incorporated realistic joint angle limits, motion frequencies for running and walking, and other practical considerations. This allows the results to closely reflect reality and provides a clear basis for explaining their underlying causes.

Efficient joint pose representation. To precisely describe the poses of the robot’s arms and legs, the standard Denavit-Hartenberg (D-H) method was used to establish coordinate frames for each joint. By deriving the homogeneous transformation matrices between adjacent frames, we achieved an efficient, chain-based representation of joint poses, allowing accurate calculation of key motion parameters such as joint rotation angles.

2.Disadvantages

Many simplifying assumptions. To facilitate computation and modeling, numerous assumptions were made to simplify the model. In real-world applications, these assumptions would need to be carefully refined based on actual conditions.

7.2 Future Work

The mathematical models established in this paper for the Yushu Technology G1 robot are not limited to the G1 robot and can be applied to other humanoid robots as well. The robotic arm control model developed in Problem 1 can be used for end-effector control of general robotic arms. The multi-joint coordinated motion model from Problem 2, combined with environmental perception devices such as the D435i depth camera, can be applied to autonomous walking, obstacle avoidance, stair climbing, and other robotic applications. The dance motion model from Problem 3 can be further extended to study the stability of humanoid robots in other scenarios and motions. The energy consumption model from Problem 4 can be widely applied across various robotic activities,

providing low-energy parameter choices to extend robot operational endurance.

7.3 Conclusion

This paper focuses on the Yushu Technology G1 robot and investigates how to (1) obtain the end-effector coordinates from robotic arm joint angles, (2) analyze the variation of each joint angle during robot motion and dance, and (3) maintain robot stability. Finally, with the goal of reducing overall energy consumption, it proposes parameter optimization strategies for lower energy usage. Specifically, the following work was carried out:

- Calculated the end-effector positions of the robotic arm based on the given joint angles
- Developed a time-dependent model of the robot's joint angles during motion, plotted the joint angle variations over time, identified the moment of maximum knee flexion, and determined the total motion duration T
- Established a model for the time-varying joint angles of the arms and legs during the robot's dance motions
- Proposed an optimized control scheme aimed at minimizing energy consumption, enabling the robot to perform actions with reduced energy expenditure

VIII. References

[1] Liang Liang, Wu Chengdong, Liu Shichang. Robot Absolute Position Accuracy Calibration Algorithm Based on Joint Geometric Error[J]. Journal of Northeastern University (Natural Science), 2025, 46(4): 1-7.

[2] Fu Haitao, Zhong Senming, Luo Zhida. Development and Application of the Compensation Algorithm for the Joint Angle of Industrial Robot[J]. Metrology and Measurement Technology, 2021, 48(10): 60-64.

[3] Qiu Binquan, Chen Silu, Gu Yingkui, Zhang Chi, Yang Guilin. Synchronous Planning of Trapezoidal Trajectory for Robot Joints with Optimal Energy Consumption[J]. Machine Design and Research, 2022, 38(4): 32-36.

[4] Zeng Qiang, Fang Yuefa. Motion plan of global coordination for biped robot[J]. Machinery, 2007, 34(2): 53-57.

[5] Unitree Robotics. About G1 Humanoid Robot[EB/OL]. `https://support.unitree.com/home/zh/G1_developer/about_G1`, 2025-11-22.

[6] Zhu Xiaoguang. Research on Gait and Path Planning for Biped Robot[D]. North China Electric Power University, 2012.

[7] Zhang Y. Research on the Key Technology for the Open Control Platform of Parallel Robots[D]. Harbin Institute of Technology, Shenzhen, 2012.

[8] Wang Hongjie. Research on Control System of Humanoid Robot Based on ROS[D]. Beijing University of Chemical Technology, 2022. bibitem2025Peng Peng Chao, Xia Kerui. A Universal S-curve Velocity Planning Algorithm with Beginning and Ending Velocity Arbitrarily Assigned[J]. Manufacturing Automation, 2025, 47(8): 47-54.

[9] Peng Chao, Xia Kerui. A Universal S-curve Velocity Planning Algorithm with Beginning and Ending Velocity Arbitrarily Assigned[J]. Manufacturing Automation, 2025, 47(8): 47-54.

[10] Ruan Xiaogang, Li Zhiqian. A Bipedal Locomotion Planning with Double Support Phase Based on Linear Inverted Pendulum Mode[C]//Global Congress on Intelligent Systems. 2009: 1-5.

[11] Wu Yongqiang, Tang Xianzhi, Song Wei, Jiang Pei, Zhou Jin, Chen Yuanjie. Power equivalent model of industrial robot and parameter identification[J]. Journal of Chongqing University, 2021, 44(10): 28-37.

[12] Naksuk N, Lee C S G. Zero Moment Point Manipulability Ellipsoid[C]//Proceedings of the 2006 IEEE International Conference on Robotics and Automation. 2006: 1942-1947.

本参赛队未使用任何 AI 工具

IX. Appendix

Listing 1: Question 1 source code

```
clc, clear, close all;
% 初始状态的参数
d = [0, 0, 0, 0];
a = [0, 0, 0.169, 0.169];
alpha = [90, 0, 0, 0];

% 直接计算末端的位置就可以
theta_b = [30, 30, 0, 0];

% 初始化变换矩阵
T = eye(4);
P = zeros(length(theta_b), 3);          % 用来存储最后节点的位置
for i = 1:length(theta_b)
T = forward_kinematics(T, d(i), a(i), alpha(i), theta_b(i));
output = T * [0;0;0;1];
P(i, :) = [output(1), output(2),output(3)];
fprintf('关节坐标为: (%.5f, %.5f, %.5f)\n',output(1), output(2),output(3));
end

% 是否满足电机约束
m1 = 2.788/2;                % 单个大臂
g = 9.8;
P1 = P(3,:);                 % 手肘处的坐标
P2 = P(4,:);                 % 手臂末端坐标
tau_stad = 120;
if safe_motor(m1, g, P1, P2, tau_stad)
fprintf('满足电机约束');
end

function T = forward_kinematics(T, d, a, alpha, theta)
% 进行一次正向运动学
A = [cosd(theta), -sind(theta)*cosd(alpha), sind(theta)*sind(alpha), a*cosd(theta);
sind(theta), cosd(theta)*cosd(alpha), -cosd(theta)*sind(alpha), a*sind(theta);
0, sind(alpha), cosd(alpha), d;
0, 0, 0, 1];
% cosd是角度的cos函数
T = T * A;
end

function flag = safe_motor(m, g, P1, P2, tau_stad)
% 判断整个腿或者手臂的电机能否在设定的功率
flag = false;
tau1 = cross(P1/2, [0,0,-m*g])+ cross(P1 + P2/2, [0,0,-m*g]); % 计算肩膀处的电机扭矩
tau2 = cross(P2/2, [0,0,-m*g]);     % 计算手肘处的电机扭矩
t1 = 0; t2 = 0;                     % 对应的模长
```

```
for i = 1:3
t1 = t1 + tau1(i)*tau1(i);
t2 = t2 + tau2(i)*tau2(i);
end
if (sqrt(t1)<=tau_stad) && (sqrt(t2)<=tau_stad)
flag = true;
end
end
```

Listing 2: Question 2 source code

```
clc, clear, close all;

% 初始状态的参数 - 统一使用弧度制
d = [0, 0, 0, 0, 0, 0];            % 连杆偏移
a = [0, 0, 0.3, 0.3, 0, 0];        % 连杆长度 (单位:米)
alpha = [pi/2, pi/2, 0, 0, 0, pi/2]; % 连杆扭角 (统一用弧度)

% 角度限制 (保持弧度制)
theta1_range = [-2.5307, 2.8798]; % L_LEG_HIP_PITCH
theta2_range = [-0.5236, 2.9671]; % L_LEG_HIP_ROLL
theta3_range = [-2.7576, 2.7576]; % L_LEG_HIP_YAW
theta4_range = [-0.087267, 2.8798]; % L_LEG_KNEE
theta5_range = [-0.87267, 0.5236]; % L_LEG_ANKLE_PITCH
theta6_range = [-0.2618, 0.2618]; % L_LEG_ANKLE_ROLL

rng(520);
v_m = pi;                          % 速度峰值
T = 5;
dt = 0.005;
t_b = 0:0.5:5;                     % 步长时间划分
t_a = 0:dt:5;                      % 求夹角划分
s = 5 * (1 - cos(pi*t_b/5));
h = 0.48;
result_l = zeros(1,length(t_a));
result_r = zeros(1,length(t_a));
% 预先设置参数
lb = [theta1_range(1), theta2_range(1), theta3_range(1), theta4_range(1),
    theta5_range(1), theta6_range(1)];
ub = [theta1_range(2), theta2_range(2), theta3_range(2), theta4_range(2),
    theta5_range(2), theta6_range(2)];

theta1_l = zeros(1,length(t_a));
theta2_l = zeros(1,length(t_a));
theta3_l = zeros(1,length(t_a));
theta4_l = zeros(1,length(t_a));
theta5_l = zeros(1,length(t_a));
theta6_l = zeros(1,length(t_a));
theta1_r = zeros(1,length(t_a));
```

```
theta2_r = zeros(1,length(t_a));
theta3_r = zeros(1,length(t_a));
theta4_r = zeros(1,length(t_a));
theta5_r = zeros(1,length(t_a));
theta6_r = zeros(1,length(t_a));

theta_opt = [0, 0, 0, pi/4, 0, 0];
for i = 1:length(t_b)-1
f = s(i+1) - s(i);          % 每一步的步长
x = f/2;                    % 一半的距离
total = 0;
for j = 1:101
t = (i-1)*0.5+(j-1)*dt;
x_now = x - (5 * (1 - cos(pi*t/5)) - s(i));
target = [x_now, 0, -h];
% 使用全局优化算法增强鲁棒性
opts = optimoptions(@fmincon,'Algorithm', 'sqp', 'MaxIterations', 1000,
    'MaxFunctionEvaluations', 3000, ...
'StepTolerance', 1e-10, 'OptimalityTolerance', 1e-10, 'Display', 'off');
% 创建优化问题

problem = createOptimProblem('fmincon', 'objective', @(theta) objective_func(theta, d,
    a, alpha, target), ...
'x0', theta_opt, 'lb', lb, 'ub', ub, 'options', opts);

% 创建全局搜索器，但减少试验点以提高效率
gs = GlobalSearch('NumTrialPoints', 400, 'MaxTime', 60, 'Display', 'off');
[theta_opt, fval, ~] = run(gs, problem);
fprintf('位置误差: %.6f m\n', fval);

total = total + fval;
H_swing = 0.6 - h;
t_swing = (j-1)*dt;
T_ssp = 0.4;
x_swing = H_swing/2*(2*pi*t_swing/T_ssp - sin(2*pi*t_swing/T_ssp)) - pi*H_swing/2;
z_swing = H_swing/2*(1 - cos(2*pi*t_swing/T_ssp));
target_swing = [x_swing, 0, z_swing];
[theta_swing, fval, ~] = run(gs, problem);
fprintf('位置误差(swing): %.6f m\n', fval);
total = total + fval;

if flag
theta1_l(1, (i-1)*100 + j) = theta_opt(1);
theta2_l(1, (i-1)*100 + j) = theta_opt(2);
theta3_l(1, (i-1)*100 + j) = theta_opt(3);
theta4_l(1, (i-1)*100 + j) = theta_opt(4);
theta5_l(1, (i-1)*100 + j) = theta_opt(5);
theta6_l(1, (i-1)*100 + j) = theta_opt(6);
result_l(1, (i-1)*100 + j) = theta_opt(4);
```

```
theta1_r(1, (i-1)*100 + j) = theta_swing(1);
theta2_r(1, (i-1)*100 + j) = theta_swing(2);
theta3_r(1, (i-1)*100 + j) = theta_swing(3);
theta4_r(1, (i-1)*100 + j) = theta_swing(4);
theta5_r(1, (i-1)*100 + j) = theta_swing(5);
theta6_r(1, (i-1)*100 + j) = theta_swing(6);
result_r(1, (i-1)*100 + j) = theta_swing(4);

else
theta1_r(1, (i-1)*100 + j) = theta_opt(1);
theta2_r(1, (i-1)*100 + j) = theta_opt(2);
theta3_r(1, (i-1)*100 + j) = theta_opt(3);
theta4_r(1, (i-1)*100 + j) = theta_opt(4);
theta5_r(1, (i-1)*100 + j) = theta_opt(5);
theta6_r(1, (i-1)*100 + j) = theta_opt(6);
result_r(1, (i-1)*100 + j) = theta_opt(4);
theta1_l(1, (i-1)*100 + j) = theta_swing(1);
theta2_l(1, (i-1)*100 + j) = theta_swing(2);
theta3_l(1, (i-1)*100 + j) = theta_swing(3);
theta4_l(1, (i-1)*100 + j) = theta_swing(4);
theta5_l(1, (i-1)*100 + j) = theta_swing(5);
theta6_l(1, (i-1)*100 + j) = theta_swing(6);
result_l(1, (i-1)*100 + j) = theta_swing(4);
end
end
disp(total);
if flag
flag = false;
else
flag = true;
end
end

figure(1)
plot(t_a, theta1_l/pi*180);
figure(2)
plot(t_a, theta2_l/pi*180);
figure(3)
plot(t_a, theta3_l/pi*180);
figure(4)
plot(t_a, theta4_l/pi*180);
figure(5)
plot(t_a, theta5_l/pi*180);
figure(6)
plot(t_a, theta6_l/pi*180);
figure(7)
plot(t_a, theta1_r/pi*180);
figure(8)
plot(t_a, theta2_r/pi*180);
```

```
figure(9)
plot(t_a, theta3_r/pi*180);
figure(10)
plot(t_a, theta4_r/pi*180);
figure(11)
plot(t_a, theta5_r/pi*180);
figure(12)
plot(t_a, theta6_r/pi*180);
figure(13)
plot(t_a, result_l/pi*180);
figure(14)
plot(t_a, result_r/pi*180);
figure(15)
plot(t_a, max(result_r,result_l)/pi*180);
%% 目标函数
function error = objective_func(theta, d, a, alpha, target)
T = forward_kinematics(d, a, alpha, theta);
actual_pos = T(1:3, 4)';
error = norm(actual_pos - target);
end

%% 正向运动学函数
function T = forward_kinematics(d, a, alpha, theta)
% 假设腿部运动学链: 髋部 -> 大腿 -> 小腿 -> 脚
% 使用标准的DH参数法
T = eye(4);

for i = 1:length(theta)
% DH变换矩阵
A = [cos(theta(i)), -sin(theta(i))*cos(alpha(i)), sin(theta(i))*sin(alpha(i)),
    a(i)*cos(theta(i));
sin(theta(i)), cos(theta(i))*cos(alpha(i)), -cos(theta(i))*sin(alpha(i)),
    a(i)*sin(theta(i));
0, sin(alpha(i)), cos(alpha(i)), d(i);
0, 0, 0, 1];

T = T * A;
end
end
```

Listing 3: Question 3 source code

```
import numpy as np
import matplotlib.pyplot as plt

class RobotModel:
def __init__(self):

self.R = 0.3
```

```
self.T = 4.0
self.d_y = 0.1
self.L_spine = 0.3


self.H_waist = 0.55

self.W_s = 0.085
self.W_hip = 0.085
self.W_step = 0.085

self.L_arm1 = 0.30
self.L_arm2 = 0.30
self.L_leg1 = 0.30
self.L_leg2 = 0.30

self.m_trunk = 19.5
self.m_u = 1.394
self.m_f = 1.394
self.M_low = 9.9
self.M_up = self.m_trunk + 2 * self.m_u + 2 * self.m_f


def rot_z(self, theta):
c, s = np.cos(theta), np.sin(theta)
return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])

def rot_y(self, theta):
c, s = np.cos(theta), np.sin(theta)
return np.array([[c, 0, s], [0, 1, 0], [-s, 0, c]])


def solve_arm_ik(self, x, y, z, is_left):

L_reach = np.sqrt(x ** 2 + y ** 2 + z ** 2)


cos_el = (self.L_arm1 ** 2 + self.L_arm2 ** 2 - L_reach ** 2) / (2 * self.L_arm1 *
    self.L_arm2)
cos_el = np.clip(cos_el, -1.0, 1.0)
theta_elbow = np.pi - np.arccos(cos_el)

theta_pitch = np.arctan2(z, x)

sign = 1.0 if is_left else -1.0
theta_yaw = np.arctan2(x, sign * self.R)

R_sh = self.rot_z(theta_yaw) @ self.rot_y(theta_pitch)
vec_upper = R_sh @ np.array([self.L_arm1, 0, 0])
```

```
return theta_pitch, theta_yaw, theta_elbow, vec_upper

def solve_leg_ik(self, P_pelvis, is_left=True):
y_sign = 1 if is_left else -1
P_hip = P_pelvis + np.array([0, y_sign * self.W_hip, 0])
P_foot = np.array([0, y_sign * self.W_step, 0])
V = P_foot - P_hip
vx, vy, vz = V[0], V[1], V[2]

# Roll
theta_roll = np.arctan2(vy, -vz)
L_sag = np.sqrt(vy ** 2 + vz ** 2)

# Pitch
L_virt = np.sqrt(vx ** 2 + L_sag ** 2)

cos_kn = (self.L_leg1 ** 2 + self.L_leg2 ** 2 - L_virt ** 2) / (2 * self.L_leg1 *
    self.L_leg2)
cos_kn = np.clip(cos_kn, -1.0, 1.0)
theta_knee = np.pi - np.arccos(cos_kn)

alpha = np.arctan2(-vx, L_sag)
cos_beta = (self.L_leg1 ** 2 + L_virt ** 2 - self.L_leg2 ** 2) / (2 * self.L_leg1 *
    L_virt)
cos_beta = np.clip(cos_beta, -1.0, 1.0)
beta = np.arccos(cos_beta)
theta_hip_p = alpha + beta

theta_ankle = -(theta_hip_p + theta_knee)
return theta_roll, theta_hip_p, theta_knee, theta_ankle

# --- 仿真主程序 ---
def run(self):
t_array = np.linspace(0, self.T, 200)
omega = np.pi / 2

res = {
   't': t_array, 'theta_waist': [], 'pelvis': [],
   'com_offset': [],
   'arm_L': [], 'arm_R': [], 'leg_L': [], 'leg_R': []
}

center_x_offset = 0.20
center_z_offset = -0.10

for t in t_array:
th_w = (np.pi / 16) * t
R_trunk = self.rot_z(th_w)
```

```
s, c = np.sin(omega * t), np.cos(omega * t)

x_L = self.R * s + center_x_offset
z_L = -self.R * c + center_z_offset
rel_L = np.array([x_L, self.d_y, z_L])

x_R = -self.R * s + center_x_offset
z_R = -self.R * c + center_z_offset
rel_R = np.array([x_R, -self.d_y, z_R])

# IK
pL, yL, eL, vL = self.solve_arm_ik(rel_L[0], rel_L[1], rel_L[2], True)
pR, yR, eR, vR = self.solve_arm_ik(rel_R[0], rel_R[1], rel_R[2], False)

C_tr_loc = np.array([0, 0, self.L_spine / 2])

P_sh_L = np.array([0, self.W_s, self.L_spine])
P_el_L = P_sh_L + vL
P_hd_L = P_sh_L + rel_L
C_Lu_loc = (P_sh_L + P_el_L) / 2
C_Lf_loc = (P_el_L + P_hd_L) / 2

P_sh_R = np.array([0, -self.W_s, self.L_spine])
P_el_R = P_sh_R + vR
P_hd_R = P_sh_R + rel_R
C_Ru_loc = (P_sh_R + P_el_R) / 2
C_Rf_loc = (P_el_R + P_hd_R) / 2

P_w_base = np.array([0, 0, self.H_waist])

def to_w(p):
return R_trunk @ p + P_w_base

C_up_W_abs = (self.m_trunk * to_w(C_tr_loc) +
self.m_u * (to_w(C_Lu_loc) + to_w(C_Ru_loc)) +
self.m_f * (to_w(C_Lf_loc) + to_w(C_Rf_loc))) / self.M_up

off_x = C_up_W_abs[0]
off_y = C_up_W_abs[1]
if abs(off_x) < 1e-4: off_x = 0.0
if abs(off_y) < 1e-4: off_y = 0.0

x_pel = - (self.M_up / self.M_low) * off_x
y_pel = - (self.M_up / self.M_low) * off_y

P_pel = np.array([x_pel, y_pel, self.H_waist])

lL = self.solve_leg_ik(P_pel, True)
```

```
lR = self.solve_leg_ik(P_pel, False)

res['theta_waist'].append(th_w)
res['com_offset'].append([off_x, off_y])
res['pelvis'].append([x_pel, y_pel])
res['arm_L'].append([pL, yL, eL])
res['arm_R'].append([pR, yR, eR])
res['leg_L'].append(lL)
res['leg_R'].append(lR)

for k in res: res[k] = np.array(res[k])
return res


def plot_res(res):
t = res['t']
fig, axs = plt.subplots(2, 2, figsize=(14, 10))
plt.subplots_adjust(hspace=0.3)

axs[0, 0].plot(t, np.degrees(res['theta_waist']), color = '#328986')
axs[0, 0].set_xlabel('Time (s)')
axs[0, 0].set_ylabel('WAIST_Angle (°)')
for spine in axs[0, 0].spines.values():
spine.set_linewidth(2)
axs[0, 0].tick_params(axis='x', direction='in', length=6, width=2)
axs[0, 0].tick_params(axis='y', direction='in', length=6, width=2)

L_pitch = np.degrees((res['arm_L'][:, 0]))*0.83
axs[0, 1].plot(t, L_pitch, color = '#546a93', label='L_Shoulder_Pitch')
axs[0, 1].plot(t, np.degrees(res['arm_L'][:, 2])-50, color = '#328986',
    label='L_Elbow')

axs[0, 1].set_xlabel('Time (s)')
axs[0, 1].set_ylabel('L_SHOULDER_Angle (°)')
axs[0, 1].legend()
for spine in axs[0, 1].spines.values():
spine.set_linewidth(2)
axs[0, 1].tick_params(axis='x', direction='in', length=6, width=2)
axs[0, 1].tick_params(axis='y', direction='in', length=6, width=2)


axs[1, 0].plot(t, np.degrees(res['leg_L'][:, 1]), label='Hip Pitch', color =
    '#546a93')
axs[1, 0].plot(t, np.degrees(res['leg_L'][:, 2]), label='Knee Pitch',color =
    '#328986')
axs[1, 0].set_xlabel('Time (s)')
axs[1, 0].set_ylabel('L_LEG_Angle (°)')
axs[1, 0].set_ylim(0, 60)
axs[1, 0].legend()
```

```
for spine in axs[1, 0].spines.values():
spine.set_linewidth(2)
axs[1, 0].tick_params(axis='x', direction='in', length=6, width=2)
axs[1, 0].tick_params(axis='y', direction='in', length=6, width=2)


axs[1, 1].plot(t, np.degrees(res['leg_R'][:, 1]), label='Hip Pitch', color =
    '#546a93')
axs[1, 1].plot(t, np.degrees(res['leg_R'][:, 2]), label='Knee Pitch', color =
    '#328986')
axs[1, 1].set_xlabel('Time (s)')
axs[1, 1].set_ylabel('R_LEG_Angle (°)')
axs[1, 1].set_ylim(0, 60)
axs[1, 1].legend()
for spine in axs[1,1].spines.values():
spine.set_linewidth(2)
axs[1, 1].tick_params(axis='x', direction='in', length=6, width=2)
axs[1, 1].tick_params(axis='y', direction='in', length=6, width=2)
plt.savefig('q_3.png', format='png')
plt.show()


if __name__ == '__main__':
model = RobotModel()
data = model.run()
plot_res(data)
```

Listing 4: Question 4 source code

```
import numpy as np
import matplotlib.pyplot as plt


class RobotSpecs:
def __init__(self):

self.L_arm = 0.30
self.L_leg = 0.35
self.m_arm = 2.5
self.m_leg = 4.5
self.m_trunk = 19.5
self.g = 9.81


self.R_coil = 0.5
self.Kt = 2.0
self.kc = self.R_coil / (self.Kt ** 2)

self.eta_drive = 0.85
```

```
self.eta_reg = 0.60



def calculate_torque_power(q, dq, ddq, joint_type, specs):


if joint_type == 'arm':
tau_g = specs.m_arm * specs.g * (specs.L_arm / 2) * np.sin(q)
inertia = specs.m_arm * (specs.L_arm ** 2) / 3
elif joint_type == 'leg':
tau_g = specs.m_leg * specs.g * (specs.L_leg / 2) * np.sin(q)
inertia = specs.m_leg * (specs.L_leg ** 2) / 3
else: # waist
tau_g = 0
inertia = 0.5 * specs.m_trunk * (0.15 ** 2)
tau_a = inertia * ddq
tau = tau_g + tau_a


mech_power = tau * dq
copper_loss = specs.kc * (tau ** 2)

if mech_power > 0:
p_elec = (mech_power / specs.eta_drive) + copper_loss
else:
p_elec = (specs.eta_reg * mech_power) + copper_loss

return tau, p_elec

def generate_full_trajectory(specs):
dt = 0.01


t1 = np.arange(0, 5.0, dt)

q_lift = np.deg2rad(90) * (1 - np.cos(np.pi * t1 / 5.0)) / 2
dq_lift = np.gradient(q_lift, dt)
ddq_lift = np.gradient(dq_lift, dt)

t2 = np.arange(0, 5.0, dt)

freq = 2.0 * np.pi * 2.0
q_walk = np.deg2rad(30) * np.sin(freq * t2)
dq_walk = np.gradient(q_walk, dt)
ddq_walk = np.gradient(dq_walk, dt)

t3 = np.arange(0, 4.0, dt)
q_waist = np.deg2rad(45) * (t3 / 4.0)
```

```
dq_waist = np.full_like(t3, np.deg2rad(45) / 4.0)
ddq_waist = np.zeros_like(t3)

omega = 2 * np.pi / 4.0
q_dance = np.deg2rad(75) * np.sin(omega * t3 - np.pi / 2) - np.deg2rad(25)
dq_dance = np.gradient(q_dance, dt)
ddq_dance = np.gradient(dq_dance, dt)

q_stat = np.full_like(t3, np.deg2rad(30))
dq_stat = np.zeros_like(t3)
ddq_stat = np.zeros_like(t3)

return (t1, q_lift, dq_lift, ddq_lift), \
(t2, q_walk, dq_walk, ddq_walk), \
(t3, q_dance, dq_dance, ddq_dance, q_waist, dq_waist, ddq_waist, q_stat, dq_stat,
    ddq_stat)

def calculate_total_energy(specs):
(d1, d2, d3) = generate_full_trajectory(specs)

time_seq = []
power_seq = []
dt = 0.01
curr_t = 0

# --- Process Q1 ---
t, q, dq, ddq = d1
for i in range(len(t)):
_, p_L = calculate_torque_power(q[i], dq[i], ddq[i], 'arm', specs)
_, p_R = calculate_torque_power(0, 0, 0, 'arm', specs)
p_legs = 2 * calculate_torque_power(0, 0, 0, 'leg', specs)[1]
p_total = p_L + p_R + p_legs
power_seq.append(p_total)
time_seq.append(curr_t + t[i])
curr_t += t[-1]

# --- Process Q2 ---
t, q, dq, ddq = d2
for i in range(len(t)):
_, p_leg = calculate_torque_power(q[i], dq[i], ddq[i], 'leg', specs)
p_total = 2 * p_leg
power_seq.append(p_total)
time_seq.append(curr_t + t[i])
curr_t += t[-1]

# --- Process Q3 ---
t, q_arm, dq_arm, ddq_arm, q_w, dq_w, ddq_w, q_l, dq_l, ddq_l = d3
for i in range(len(t)):
_, p_arm = calculate_torque_power(q_arm[i], dq_arm[i], ddq_arm[i], 'arm', specs)
```

```
_, p_waist = calculate_torque_power(q_w[i], dq_w[i], ddq_w[i], 'waist', specs)
_, p_leg = calculate_torque_power(q_l[i], dq_l[i], ddq_l[i], 'leg', specs)
p_total = 2 * p_arm + p_waist + 2 * p_leg
power_seq.append(p_total)
time_seq.append(curr_t + t[i])

t_arr = np.array(time_seq)
p_arr = np.array(power_seq)

e_arr = np.cumsum(p_arr) * dt
total_e = e_arr[-1]

return t_arr, p_arr, e_arr, total_e

def plot_results(t, p, total_e):
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(t, p, 'b-', linewidth=1.5, label='Instantaneous Power')
ax.fill_between(t, p, alpha=0.1, color='blue')

ax.axvline(x=5.0, color='r', linestyle='--')
ax.axvline(x=10.0, color='r', linestyle='--')

ax.text(2.5, max(p) * 0.9, 'Q1: Pose (Static)', ha='center', fontsize=12,
    fontweight='bold')
ax.text(7.5, max(p) * 0.9, 'Q2: Walk (Dynamic)', ha='center', fontsize=12,
    fontweight='bold')
ax.text(12.0, max(p) * 0.9, 'Q3: Dance (Optimized)', ha='center', fontsize=12,
    fontweight='bold')

ax.set_title(f'Whole-Process Power Consumption Analysis\nTotal Energy =
    {total_e:.2f} Joules', fontsize=14)
ax.set_xlabel('Time (s)', fontsize=12)
ax.set_ylabel('Power (W)', fontsize=12)
ax.grid(True)
ax.legend()

plt.tight_layout()
plt.show()

if __name__ == "__main__":
specs = RobotSpecs()
specs.kc = 0.0
t, p, e, total = calculate_total_energy(specs)

plot_results(t, p, total)
```